WAKARU

November 12, 2026

# Unsupervised Steering Vectors from Learned Classifiers: A Summary

Keira Wiechecki
Wakaru · kaw504@nyu.edu

Lamb Wakefield
Flock Capital · lamb@flock.holdings

Jade Zaslavsky
Wakaru · jadeofnameless@gmail

vmfunc
Flock Capital · celeste@linux.com

Auditing a model for a hidden goal or an unnamed concept requires identifying its internal variables without labeled examples. This summary presents an unsupervised method designed for that task and summarizes its current empirical standing. The method compresses a network's internal state into a discrete categorical description that preserves nearly all of the network's downstream performance. However, editing that description steers the network no more specifically than swapping between arbitrary categories of the same representation, and past the first round of the code, independent training runs learn different categories. Until the learned variables are both reproducible across runs and more causally specific than arbitrary partitions, they cannot be reliably named or used in an audit.

## What the method does

The method, which we call an *Ontologizer*, encodes a neural network's internal representation at a given layer as a short list of discrete categorical choices and reconstructs the representation from those choices. Each round of the encoder consists of a bank of small classifiers, each of which assigns every input to one of $k$ categories associated with a learned vector. The representation is reconstructed by summing the vectors of the selected categories. In the main GPT-2 models described below, the encoder stacks 5 rounds of 32 classifiers with 32 categories each, describing each input with 160 discrete choices (800 bits) and one scalar per round that scales that round's contribution. Training is entirely unsupervised: the classifiers and category vectors are learned solely to minimize reconstruction error.

Because the categories within each classifier are mutually exclusive, changing a classifier's assignment from category $j$ to category $j'$ replaces one learned vector with another. This category swap is the basic steering intervention of the architecture, shifting the representation from $j$ to $j'$ along a learned contrastive axis. Every swap produces another valid discrete code in which each classifier still selects one of its categories. However, a valid code is not guaranteed to lie within the distribution of codes the model produces on natural inputs—for instance, if two

classifiers only ever select certain categories together, changing one without the other creates an out-of-distribution combination—so whether steered representations remain well-behaved must be tested empirically.

We evaluate the method on two representations. The primary testbed is the residual stream of GPT-2 small after its eighth transformer block [1] on web text, where reconstructed or edited states can be spliced directly back into the forward pass to measure how much of the model's next-token prediction performance is preserved. The second testbed is SONAR sentence embeddings [2], which represent the opening of each web document as a single fixed-length vector that can be decoded back into text to inspect the effect of steering.

## What it is for

Behavioral evaluation alone cannot establish what a model is optimizing for, because a model can behave differently when it infers that it is being trained or evaluated [3]. Auditing a model's goals therefore requires inspecting its internal representations, including concepts that an evaluator has not thought to name in advance.

In activation steering, the goal is to modify a model's internal state so as to change its behavior in a controlled direction. A standard supervised approach is the *difference-of-means* steering vector, computed by averaging the representations of examples with a target property, subtracting the average of examples without it, and adding that difference vector to new activations [4]; in some cases even a single contrastive pair of prompts suffices [5]. While supervised steering vectors and linear probes are effective, both require contrastive examples of a property that an experimenter already suspects is present. They can therefore test for a hypothesized concept, but cannot discover an unsuspected one.

Sparse autoencoders (SAEs) learn features without supervision by representing each input as a sparse combination of a few active feature directions out of thousands [6, 7]. However, each SAE feature is an isolated direction that is either active or inactive rather than a choice among explicit alternatives. When a narrow feature implies a broader one, the narrow feature can absorb the broader feature's direction so that the broader feature stays silent even when its concept is present [8], making inactivity weak evidence that a property is absent. SAE features also split into finer variants as dictionary size increases [9], and adding an unbounded multiple of a feature direction can push activations off the natural manifold. On a recent steering benchmark, prompting, fine-tuning, and supervised difference-of-means vectors all outperformed SAE features [10].

The supervised steering methods that perform best work by contrasting alternatives—either contrasting the presence of a property with its absence, or contrasting one value of a categorical *variable* (such as a document's topic or language) with another. Our goal is to learn unsupervised steering vectors that share this contrastive structure, representing each variable as a set of mutually exclusive alternatives so that swapping between categories applies a contrastive difference vector. Testing whether such swaps identify the model's actual variables requires comparing them against a null baseline: a category swap demonstrates that a learned variable is causally meaningful only if it steers the model more specifically than swaps between arbitrary

partitions of the same representation space. On both GPT-2 and SONAR, the learned categories do not yet beat this baseline.

## What it assumes, and why that is acceptable

1. **A representation can be approximated as a sum of discrete categorical choices refined across successive residual rounds.** Many properties of language (token identity, syntax, topic) are naturally categorical, and iterative residual refinement is the standard mechanism of residual vector quantization [11, 12]. On GPT-2, an 800-bit code per token reconstructs held-out activations accurately enough that splicing the reconstruction into the forward pass recovers 97.5% of the next-token cross-entropy loss relative to mean ablation.

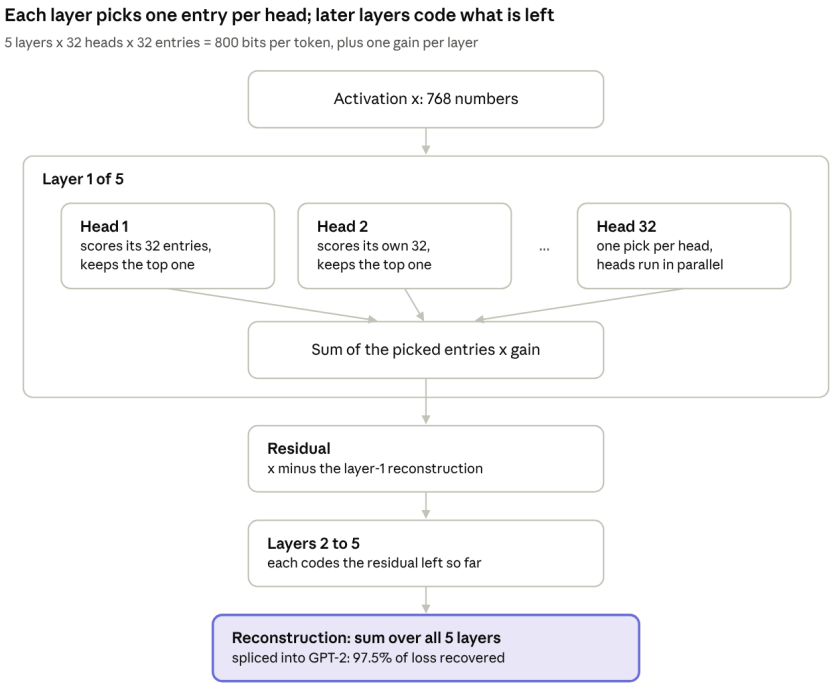
2. **Classifiers within a round act as roughly independent factors.** Combinatorial expressiveness comes from combining largely independent choices across classifiers. On GPT-2, pairs of classifiers share an average of 0.27 bits of mutual information in the first round and 0.02 bits by the third round, out of roughly 5 bits of entropy per classifier.

3. **The network continues to function when run on the reconstructed state.** Steering through the discrete code is only meaningful if the reconstruction preserves normal model computation. Beyond the 97.5% loss recovery of the full reconstruction, swapping a single classifier's choice in the first round perturbs GPT-2's next-token distribution by only 0.001–0.06 nats.

4. **Learned categories correspond to nameable concepts.** This holds only partially. On GPT-2, when an automated LLM judge is given a natural-language description of a first-round category, it distinguishes the category's highest-margin examples from random distractors with 0.91 balanced accuracy (chance 0.50), but accuracy drops to 0.65 on random category members and 0.55 near the category boundary, and falls to chance in the fourth and fifth rounds even on typical examples. Moreover, 29–31 of the 32 first-round classifiers primarily track current-token identity: a simple lookup table based only on the current token outperforms the LLM judge on single first-round categories, scoring 0.75 vs. 0.69 when each classifier writes to its own coordinate block and 0.74 vs. 0.71 when classifiers write *direct vectors* in the full space. On SONAR sentence embeddings, the largest first-round classifier partitions documents by broad topic and genre (such as crime news, legislation, and encyclopedic articles).

5. **Learned categories reflect the data rather than the random seed.** This assumption currently fails: two training runs initialized with different random seeds reach the same reconstruction error, but on both GPT-2 and SONAR only the first round's classifiers show above-chance agreement across runs, while deeper rounds learn different partitions.

## How it works, intuitively




Intuitively, the encoder describes an activation vector by answering a fixed bank of multiple-choice questions in which both the questions and the vector answers are learned. In the first round, 32 classifiers run in parallel and each assigns the input to one of its 32 categories; the selected category vectors are summed and scaled to match the norm of the input. The second round receives the residual error left unexplained by the first round and encodes that remainder in the same way, repeating for five rounds in total. Training uses discrete argmax choices in the forward pass and soft probabilities to propagate gradients in the backward pass (a straight-through estimator). From a compression standpoint, each round is an additive multi-codebook quantizer [13] stacked into a multi-stage residual quantizer [14] (reducing to product quantization [15] when each classifier writes to a disjoint block of coordinates). Here, however, the quantizer is used not for storage compression, but to expose discrete choices that can be inspected and swapped; a related residual quantizer has been used to discretize SONAR embeddings for concept-level language modeling [16].

## How it compares with existing methods

**Comparison with supervised steering.** We have evaluated three steering comparisons against supervised vectors so far, though the main benchmark on externally labeled concepts remains to be run:

- *Next-token specificity against arbitrary categories (GPT-2).* Swapping a token's category from $j$ to $j'$ shifts GPT-2's next-token predictions toward the empirical token distribution of

category $j'$: among the 31 alternative categories, $j'$ ranks at 3.5–6 on average at a unit step and 5–8 at four times that step, compared to 14–18 for a random step of equal norm (evaluated on one classifier each in the first, third, and fifth rounds). However, arbitrary categories formed by binning activations along 32 random directions per classifier steer just as specifically across all five rounds (mean target rank 0.21–1.69 vs. 0.56–1.37 for learned categories across 8 classifiers per round). High rank specificity therefore reflects the near-linearity of the downstream network rather than semantic structure in the learned categories.

- *Language steering (SONAR sentence embeddings).* In the soft-assignment variant of the model, forcing a category associated with a target language moves decoded text 0.64 times as far along the supervised difference-of-means direction as supervised steering itself, and 4 times as far as a linear probe direction, while all methods (including supervised steering) disturb unrelated content about as much as a random step of the same size.

- *Reaching the model's own categories (SONAR sentence embeddings).* For each category, we also fitted supervised difference-of-means and logistic-probe directions to the model's own category assignments at matched step size. In the soft model, stepping along a category's decoded vector reaches that category on 75% of inputs (vs. 46–47% for the supervised directions), whereas in the hard five-round model the decoded vector reaches its target on 37% of inputs (vs. 91–94% for the supervised directions). Again, however, arbitrary partitions are equally reachable: on an untrained five-round model, supervised vectors reach 75% and 94% of the random categories (and classifier gradients reach 61–85%) while flipping one third as many other classifiers as in the trained model (22% vs. 66%). Similarly, in a single-round bank of 380 random-direction classifiers ($k$=32), swaps between empirical category centroids reach 78% of targets and supervised or projection directions reach 87–100% while flipping 30–32% of other classifiers.

A decisive comparison requires steering toward externally defined concepts at tuned intervention strengths, following the AxBench protocol [10], where supervised difference-of-means vectors, SAE features, and category swaps are scored by the same judge on concept presence, task preservation, and fluency.

**Comparison with SAEs on steering.** On SONAR sentence embeddings at each model's native step size, stepping along an active feature of a standard SAE (32 active features per input) activates that feature on 46–50% of inputs, compared to 8–19% for the five-round hard Ontologizer, with both models disturbing unrelated features or classifiers about as much as a random step of the same norm. A single-round linear variant of the hard method reaches its target category on 41% of inputs while flipping only 1.4% of other classifiers (vs. 18% for a random step). On GPT-2, replacing a first-round classifier's choice with its mean category increases next-token loss by 0.02–0.075 nats, comparable to the 0.055-nat cost of ablating an active SAE feature.

**Comparison with SAEs on reconstruction.** Because steering through a discrete representation requires the model to function on that representation, reconstruction fidelity serves as a precondition rather than an end in itself.

| GPT-2 small, after 8 blocks, held out | per-token code | unexplained variance | loss recovered |
|---|---|---|---|
| Ontologizer, 32 categories (10M) | 800 bits + 5 numbers | 0.217 | 97.0% |
| Ontologizer, 32 categories (40M) | 800 bits + 5 numbers | 0.195 | 97.6% |
| Ontologizer, 32 categories, concat. | 800 bits + 5 numbers | 0.186 | 97.5% |
| Ontologizer, 64 categories, concat. | 960 bits + 5 numbers | 0.164 | 97.9% |
| SAE, 32 active features | 32 indices + 32 numbers | 0.189 | 96.2% |
| SAE, 128 active features | 128 indices + 128 numbers | 0.116 | 98.6% |

On GPT-2, the 800-bit discrete code recovers more of the model's next-token loss than a standard 32-feature SAE, both on the 10M-token training cache (97.0% vs. 96.2%) and after one pass over 40M tokens (97.5–97.6%), indicating that its reconstruction errors fall in directions that affect next-token predictions less. However, when the SAE's 32 active coefficients are quantized to roughly 5 bits each (556–588 total bits), the SAE achieves lower unexplained variance per bit. In the concatenated variant (`concat.` in the table), where each classifier writes to a disjoint coordinate block, replacing the shared output projection with separate per-round projections reduces unexplained variance from 0.186 to 0.169 (97.3% to 97.7% loss recovered) and reduces cross-round coupling between co-indexed classifiers from 97% to 6%. On SONAR sentence embeddings, a 1900-bit code leaves 15% of variance unexplained with per-round scalar gains and 16% when those gains are fixed to their means. At the same 1900-bit budget, a standard 160-feature SAE spends roughly 1200 bits to index its active features and leaves at least 24% of variance unexplained under any quantization of its 160 magnitudes. By contrast, a weakly penalized L1 SAE whose quantized magnitudes are entropy-coded reaches 13% unexplained variance at an average rate of 1900 bits, because its skewed firing rates make active feature indices cheaper to transmit than the Ontologizer's near-uniform categories; on a second sentence-split SONAR cache, a wider standard SAE with 256 active features and entropy coding similarly reaches 13% unexplained variance against 16% for the hard code.

## What we can claim

Across both GPT-2 and SONAR, the Ontologizer compresses internal activations into a compact set of discrete choices with reconstruction accuracy competitive with moderate-width sparse autoencoders: at a matched bit budget it outperforms standard SAEs up to 160 active features, though wider or entropy-coded SAEs achieve lower distortion. On GPT-2, splicing the discrete reconstruction into the model preserves 97.5–97.9% of next-token loss recovery, and swapping a category shifts predictions specifically toward the target category's distribution, though swaps between arbitrary random-projection categories are equally specific. A subset of first-round categories can be described in natural language: on GPT-2, an LLM judge identifies typical members of first-round categories at 0.91 balanced accuracy, though a token-identity lookup rule outperforms the judge on single categories (0.74–0.75 vs. 0.69–0.71) and accounts for most of the accuracy gain from pairing two categories in the concatenated model (0.69 to 0.78, vs. 0.71 to 0.72 with direct vectors). On SONAR, a first-round topic-and-genre classifier emerges consistently across training runs. Furthermore, adding a paired-sentence objective on SONAR

makes a designated block of 38 classifiers reproduce across three random seeds: in a one-round model trained on adjacent sentences from the same document, the 38 slow classifiers match across seeds at 0.41 normalized mutual information (vs. 0.005 for unrelated partitions), remain reproducible within individual languages, and vary more slowly across a document than any linear projection of the raw embeddings, at a cost of roughly 2% in unexplained variance. What reproduces is the block as a whole: within one run the 38 classifiers largely overlap with one another, so no single classifier is yet matched to its counterpart in another run. Whether the learned categories are more interpretable overall than SAE features, whether they can surpass arbitrary partitions on causal specificity, and whether they outperform supervised steering vectors on external concepts remain open questions.

## What does not work yet

- **Classifiers past the first round are not reproducible across seeds.** On both GPT-2 and SONAR, independent runs reach the same reconstruction error while agreeing on almost no classifiers after the first round. High reconstruction accuracy therefore does not guarantee a unique decomposition. Because non-reproducible classifiers cannot be stably named, relied upon in an audit, or used to specify a training goal that survives retraining, cross-run reproducibility is the primary bottleneck of the method, with paired-sentence training providing a first, partial mitigation.
- **Automated descriptions capture category extremes rather than ordinary members.** As noted in prior work on automated interpretability when explanations are generated from top-activating examples [17], descriptions match a category's highest-margin examples much better than random or boundary members, and categories in the fourth and fifth rounds cannot yet be described above chance despite having measurable effects on model outputs.
- **Category swaps do not yet outperform arbitrary partitions.** On both GPT-2 and SONAR, swaps between random-projection categories or categories from an untrained model steer their own cells just as specifically as swaps between learned categories, leaving reconstruction fidelity as the only demonstrated contribution of the deeper rounds so far.
- **Steering through the input disturbs other classifiers.** On SONAR embeddings, stepping in input space toward a target category flips unrelated classifiers about as often as a random step of equal norm, whereas editing the discrete code directly modifies only the chosen classifier.

## Next

Our first priority is establishing cross-run reproducibility across a broader set of classifiers. Identifiability theory shows that reconstructing marginal inputs alone cannot uniquely recover discrete or continuous internal variables without structural constraints matched to the data-generating process or paired observations such as adjacent sentences [18, 19, 20, 21, 22, 23,

24, 25, 26]. On 44.1M consecutive SONAR sentence embeddings, a linear predictor from one sentence explains only 10.4% of the next sentence's variance (unexplained variance 0.896) even though adjacent sentences are strongly aligned in cosine similarity (0.923 AUC), and classifiers trained on single sentences switch categories on 92.2–98.1% of adjacent pairs. While directly penalizing category switches across adjacent sentences collapses the penalized classifiers to 0.04–0.09 bits of entropy, maximizing a lower bound on the mutual information between a classifier's choices on adjacent sentences prevents collapse: the resulting 38 slow classifiers maintain their category across adjacent sentences well above chance (Cohen's kappa 0.46 vs. 0.07 without the objective) and reproduce as a group across three seeds. Penalizing dependence between the slow classifiers, so that each could be matched across runs on its own, matches exactly one of them and strips the rest. Immediate next steps are separating them without that loss, extending the objective to sentence triples to test three-view identifiability conditions [20], characterizing what the slow classifiers encode beyond language, and applying the paired-view objective to a language model across conversation turns.

Second, we plan to run an AxBench-style evaluation [10] comparing supervised difference-of-means vectors, SAE features, and category swaps on externally defined concepts at tuned intervention strengths, scoring concept presence, task preservation, and fluency with a shared judge and comparing concept detection against linear probes. Initial training runs for this evaluation are underway on Gemma 3 1B and Gemma 4 A4B using mean-squared error, a small category-uniformity penalty, and separate output projections for each round.

## References


[1] Alec Radford et al. *Language Models are Unsupervised Multitask Learners*. Tech. rep. OpenAI, 2019.

[2] Paul-Ambroise Duquenne, Holger Schwenk, and Benoît Sagot. *SONAR: Sentence-Level Multimodal and Language-Agnostic Representations*. 2023. arXiv: 2308.11466 [cs.CL]. URL: https://arxiv.org/abs/2308.11466.

[3] Ryan Greenblatt et al. *Alignment Faking in Large Language Models*. 2024. arXiv: 2412.14093 [cs.AI]. URL: https://arxiv.org/abs/2412.14093.

[4] Nina Rimsky et al. "Steering Llama 2 via Contrastive Activation Addition". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 15504–15522. DOI: 10.18653/v1/2024.acl-long.828. URL: https://aclanthology.org/2024.acl-long.828/.

[5] Alexander Matt Turner et al. *Steering Language Models With Activation Engineering*. 2024. arXiv: 2308.10248 [cs.CL]. URL: https://arxiv.org/abs/2308.10248.

[6] Trenton Bricken et al. "Towards Monosemanticity: Decomposing Language Models With Dictionary Learning". In: *Transformer Circuits Thread* (2023). URL: https://transformer-circuits.pub/2023/monosemantic-features/index.html.

[7] Leo Gao et al. *Scaling and evaluating sparse autoencoders*. 2024. arXiv: 2406.04093 [cs.LG]. URL: https://arxiv.org/abs/2406.04093.

[8] David Chanin et al. “A is for Absorption: Studying Feature Splitting and Absorption in Sparse Autoencoders”. In: *Advances in Neural Information Processing Systems*. Vol. 38. 2025, pp. 91489–91526. arXiv: 2409.14507 [cs.CL]. URL: https://arxiv.org/abs/2409.14507.

[9] Bart Bussmann et al. “Learning Multi-Level Features with Matryoshka Sparse Autoencoders”. In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025, pp. 6077–6101. arXiv: 2503.17547 [cs.LG]. URL: https://arxiv.org/abs/2503.17547.

[10] Zhengxuan Wu et al. “AxBench: Steering LLMs? Even Simple Baselines Outperform Sparse Autoencoders”. In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025, pp. 67035–67080. arXiv: 2501.17148 [cs.CL]. URL: https://arxiv.org/abs/2501.17148.

[11] Biing-Hwang Juang and A. H. Gray Jr. “Multiple Stage Vector Quantization for Speech Coding”. In: *ICASSP ’82. IEEE International Conference on Acoustics, Speech, and Signal Processing*. Vol. 7. 1982, pp. 597–600. DOI: 10.1109/ICASSP.1982.1171604.

[12] Christopher F. Barnes, Syed A. Rizvi, and Nasser M. Nasrabadi. “Advances in Residual Vector Quantization: A Review”. In: *IEEE Transactions on Image Processing* 5.2 (1996), pp. 226–262. DOI: 10.1109/83.480761.

[13] Artem Babenko and Victor Lempitsky. “Additive Quantization for Extreme Vector Compression”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2014, pp. 931–938. URL: https://openaccess.thecvf.com/content_cvpr_2014/html/Babenko_Additive_Quantization_for_2014_CVPR_paper.html.

[14] Neil Zeghidour et al. *SoundStream: An End-to-End Neural Audio Codec*. 2021. arXiv: 2107.03312 [cs.SD]. URL: https://arxiv.org/abs/2107.03312.

[15] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. “Product Quantization for Nearest Neighbor Search”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33.1 (2011), pp. 117–128.

[16] LCM team et al. *Large Concept Models: Language Modeling in a Sentence Representation Space*. 2024. arXiv: 2412.08821 [cs.CL]. URL: https://arxiv.org/abs/2412.08821.

[17] Gonçalo Paulo et al. “Automatically Interpreting Millions of Features in Large Language Models”. In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025. arXiv: 2410.13928 [cs.LG]. URL: https://arxiv.org/abs/2410.13928.

[18] Francesco Locatello et al. “Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations”. In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research. PMLR, 2019, pp. 4114–4124. arXiv: 1811.12359 [cs.LG]. URL: https://proceedings.mlr.press/v97/locatello19a.html.

[19] Christopher J. Hillar and Friedrich T. Sommer. *When Can Dictionary Learning Uniquely Recover Sparse Data from Subsamples?* 2015. arXiv: 1106.3616 [q-bio.NC]. URL: https://arxiv.org/abs/1106.3616.

[20] Elizabeth S. Allman, Catherine Matias, and John A. Rhodes. "Identifiability of Parameters in Latent Structure Models with Many Observed Variables". In: *The Annals of Statistics* 37.6A (2009), pp. 3099–3132. DOI: 10.1214/09-AOS689. arXiv: 0809.5032 [math.ST]. URL: https://arxiv.org/abs/0809.5032.

[21] Sunny Duan et al. "Unsupervised Model Selection for Variational Disentangled Representation Learning". In: *International Conference on Learning Representations*. 2020. arXiv: 1905.12614 [cs.LG]. URL: https://arxiv.org/abs/1905.12614.

[22] Aapo Hyvärinen and Hiroshi Morioka. "Nonlinear ICA of Temporally Dependent Stationary Sources". In: *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*. Vol. 54. Proceedings of Machine Learning Research. PMLR, 2017. URL: https://proceedings.mlr.press/v54/hyvarinen17a.html.

[23] David Klindt et al. "Towards Nonlinear Disentanglement in Natural Data with Temporal Sparse Coding". In: *International Conference on Learning Representations*. 2021. arXiv: 2007.10930 [stat.ML]. URL: https://arxiv.org/abs/2007.10930.

[24] Francesco Locatello et al. "Weakly-Supervised Disentanglement Without Compromises". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020. arXiv: 2002.02886 [cs.LG]. URL: https://arxiv.org/abs/2002.02886.

[25] Ilyes Khemakhem et al. "Variational Autoencoders and Nonlinear ICA: A Unifying Framework". In: *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics*. Vol. 108. Proceedings of Machine Learning Research. PMLR, 2020. arXiv: 1907.04809 [stat.ML]. URL: https://arxiv.org/abs/1907.04809.

[26] Roland S. Zimmermann et al. "Contrastive Learning Inverts the Data Generating Process". In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research. PMLR, 2021. arXiv: 2102.08850 [cs.LG]. URL: https://arxiv.org/abs/2102.08850.